

Application web de transfert et gestion de fichiers vers AWS S3 avec métadonnées dans DynamoDB

Contexte :

On souhaite développer une application web en **HTML / CSS / JavaScript** qui permettra à un utilisateur de télécharger un fichier depuis son navigateur et de l'envoyer à une API exposée via **Amazon API Gateway**.

Cette API va invoquer une fonction **AWS Lambda** qui :

1. Ajouter le fichier dans un **bucket Amazon S3**.
2. Récupère les métadonnées suivantes :
 - Nom du fichier
 - Taille du fichier (en octets ou format lisible)
 - Lien public du fichier dans S3
 - Type MIME du fichier (ex : `image/png`, `application/pdf`)
3. Stocker ces métadonnées dans une table **Amazon DynamoDB**.

L'application web devra ensuite afficher la liste de tous les fichiers présents (issus de DynamoDB) sous forme d'un tableau, contenant pour chaque objet :

- **Nom du fichier**
- **Taille**
- **Lien vers S3** (cliquable pour télécharger)
- **Type du fichier**

Spécifications fonctionnelles :

Front-end (HTML / CSS / JS): hébergé sur S3

- Interface simple avec :
 - Un bouton **"Choisir un fichier"**.
 - Un bouton **"Envoyer"**.
 - Un tableau listant les fichiers déjà stockés, mis à jour automatiquement après envoi.
- Utilisation de `fetch()` pour envoyer le fichier en **POST** à l'API Gateway.
- Après un upload réussi, requête GET vers l'API Gateway pour récupérer la liste complète depuis DynamoDB et mettre à jour l'interface.

Back-end (AWS)

- **Amazon API Gateway (HTTP API) :**
 - Endpoint **POST /upload** pour recevoir un fichier et le transmettre à Lambda.
 - Endpoint **GET /files** pour retourner toutes les métadonnées stockées dans DynamoDB.

- **AWS Lambda (Node.js ou Python) :**
 - **POST /upload :**
 - Récupérer le fichier depuis la requête.
 - Le sauvegarde dans S3 (avec un nom unique si nécessaire).
 - Extraire nom, taille, type MIME, lien public.
 - Insérer ces infos dans DynamoDB.
 - **GET /files :**
 - Lit DynamoDB et renvoie un tableau JSON contenant toutes les entrées.
- **Amazon S3 :**
 - Bucket configuré pour accepter des uploads depuis Lambda.
 - Objets éventuellement en accès public (ou générer des URL signées).
- **Amazon DynamoDB :**
 - Table avec clés :
 - **fileName** (clé primaire)
 - **fileSize**
 - **fileType**
 - **fileUrl**

Spécifications techniques :

- Front-end 100% statique (HTML, CSS, JS vanilla).
- Communication avec API Gateway via **JSON** et éventuellement **multipart/form-data** pour l'upload.
- Lambda avec rôles IAM autorisant **PutObject** sur S3 et **PutItem / Scan** sur DynamoDB.
- Stockage des fichiers originaux dans S3, stockage des métadonnées uniquement dans DynamoDB.

